

SpécimenManager

Document de Conception Technique Complète
Complete Technical Architecture & Design Document

v1.0 • Juin 2026

Bilingue FR / EN

PROJET / PROJECT

SpécimenManager

CLIENT

Institut Pasteur Madagascar (IPM)

STACK

PERN — PostgreSQL · Express · React · Node.js

DATE

28 Juin 2026

VERSION

1.0 — Architecture initiale + évolutions

CONFIDENTIEL — Usage interne IPM uniquement / CONFIDENTIAL — Internal use only

Table des Matières / Table of Contents

1. Architecture Globale / Global Architecture

2. Stack Technique Recommandée / Technical Stack

3. Modélisation des Données / Data Modeling

4. Architecture de l'API / API Architecture

5. Sécurité & Authentification / Security & Auth

6. Scalabilité & Déploiement / Scalability & Deployment

Synthèse & Roadmap / Summary & Roadmap

1. Architecture Globale

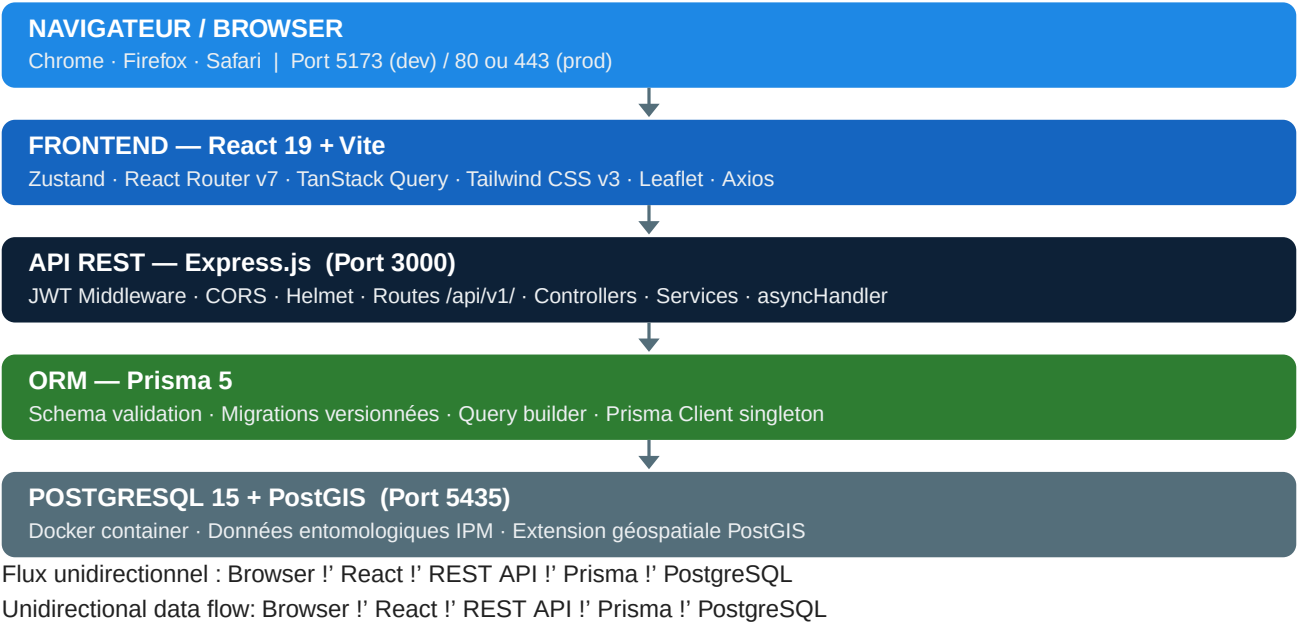
1. Global Architecture

1.1 Vue d'ensemble / Overview

SpécimenManager adopte une architecture en couches (Layered Architecture) avec le pattern MVC, déployée en mode client-serveur. Cette approche garantit une séparation claire des responsabilités, une maintenabilité optimale et une courbe d'apprentissage réduite pour les futurs développeurs.

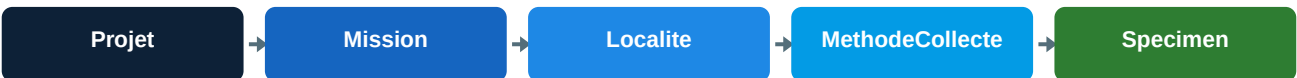
SpécimenManager adopts a layered architecture with MVC pattern in a client-server deployment. This ensures clear separation of concerns, optimal maintainability, and a reduced learning curve for future developers.

1.2 Architecture en couches / Layered Architecture Diagram



1.3 Hiérarchie métier / Business Data Hierarchy

La chaîne de containment stricte du domaine métier :
The strict domain containment chain:



Les specimens (Moustique, Tique, Puce) sont toujours reliés à une MethodeCollecte — jamais directement à une Localite ou Mission.

Specimens (Mosquito, Tick, Flea) are always linked to a MethodeCollecte — never directly to a Localite or Mission.

1.4 Patterns architecturaux retenus / Retained Architectural Patterns

Pattern	Justification FR	Justification EN
MVC (backend)	Separation Controller / Service / Model	Separation of Controller/Service/Model
RESTful API	Contrat HTTP clair, sans etat	Clear stateless HTTP contract
Repository (Prisma)	Abstraction de la persistance	Persistence abstraction layer
Singleton (Prisma client)	Evite les connexions multiples	Prevents connection pool exhaustion
Protected Routes (React)	Garde applicative cote client	Client-side application guard
Observer (TanStack Query)	Cache et invalidation automatique	Automatic cache & invalidation
SSE (Server-Sent Events)	Notifications temps reel push	Real-time server push notifications
Audit Trail	Tracabilite de toutes les mutations	Full traceability of all mutations

2. Stack Technique Recommandee

2. Recommended Technical Stack

2.1 Stack Actuelle / Current Stack

Couche	Technologie	Version	Role / Purpose
Frontend	React	19	UI framework — composants reactifs
Frontend	Vite	5.x	Bundler ultra-rapide / Ultra-fast bundler
Frontend	Tailwind CSS	v3	Utility-first CSS — design system
Frontend	React Router	v7	Routing SPA + layouts imbriquées
Frontend	TanStack Query	v5	Server-state, cache, invalidation auto
Frontend	Zustand	4.x	Global state léger / Light global state
Frontend	Leaflet	1.9	Cartes GPS interactives
Frontend	Axios	1.x	Client HTTP + intercepteurs JWT
Backend	Node.js	22 LTS	Runtime serveur / Server runtime
Backend	Express.js	4.x	Framework HTTP REST
Backend	Prisma ORM	5.x	ORM + migrations + type safety
Backend	ExcelJS	4.x	Import/Export Excel specimens
Backend	jsonwebtoken	9.x	Authentification JWT sans état
Base de données	PostgreSQL	15	SGBDR principal / Primary RDBMS
Base de données	PostGIS	3.x	Extension géospatiale / Geospatial
Infrastructure	Docker	24.x	Conteneurisation / Containerization
Infrastructure	Nginx	1.25	Reverse proxy + SSL termination

2.2 Evolutions Recommandees / Recommended Future Evolutions

Ces évolutions couvrent les besoins futurs : mobile terrain, scalabilité, robustesse production.

Besoin / Need	Technologie suggeree	Priorite
App mobile terrain offline	React Native + Expo + WatermelonDB (SQLite)	HAUTE
Sync offline vers serveur	Queue de mutations + retry automatique	HAUTE
Tests unitaires backend	Jest + Supertest	HAUTE
Tests composants frontend	Vitest + Testing Library	MOYENNE
Monitoring & erreurs	Sentry + Prometheus + Grafana	MOYENNE
CI/CD pipeline	GitHub Actions — build, test, deploy	MOYENNE
Cache API	Redis (sessions, rate-limit, invalidation)	MOYENNE
Stockage photos specimens	MinIO (auto-heberge, S3-compatible)	OPTIONNELLE
Analyse statistique	Python FastAPI micro-service + Pandas	OPTIONNELLE
WebSocket bi-directionnel	Socket.io (remplace SSE)	OPTIONNELLE

2.3 Outils de developpement / Development Tooling

Outil / Tool	Usage
ESLint + Prettier	Linting et formatage uniforme / Uniform linting & formatting
Prisma Studio	Navigateur visuel BDD localhost:5555 / Visual DB browser
Prisma Migrate	Versioning et application des migrations SQL
Docker Compose	Orchestration locale PostgreSQL / Local DB orchestration
Git + GitHub	Versionning + revues de code / Source versioning + reviews
Claude Code	Assistance developpement et documentation / Dev AI assistance

3. Modelisation des Donnees

3. Data Modeling

3.1 Entites principales / Main Entities

Le schema Prisma definit toutes les entites ci-dessous avec id autoincrement, timestamps createdAt/updatedAt, et relation vers l'utilisateur createur.

Entite / Entity	Table SQL	Description
User	users	Utilisateurs avec roles hierarchiques / Users with hierarchical roles
AuditLog	audit_logs	Journal de toutes les actions / Audit trail for all actions
Projet	projets	Projet de recherche de haut niveau / Top-level research project
Mission	missions	Campagne terrain d'un projet / Field campaign within a project
Localite	localites	Site géographique GPS + PostGIS / GPS geographic site
MethodeCollecte	methodes_collecte	Methode et dispositif de collecte / Collection method
Hote	hotes	Animal hote (tiques/puces) / Host animal (ticks/fleas)
Moustique	moustiques	Specimen Culicidae
Tique	tiques	Specimen Ixodidae / Argasidae
Puce	puces	Specimen Siphonaptera

3.2 Systeme de roles / Role Hierarchy

La hierarchie de roles est encodee numeriquement. requireMinRole() permet des verifications ascendantes.

Role	Niveau / Level	Droits FR	Droits EN
lecteur	1	Lecture seule	Read-only access
terrain	2	Saisie specimens + consultation	Specimen entry + read
chercheur	3	CRUD complet + import/export Excel	Full CRUD + Excel import/export
admin	4	Gestion utilisateurs + activation	User management + activation

Les nouveaux comptes sont crees avec actif: false — activation requise par un admin.

New accounts are created with actif: false — activation required by an admin.

3.3 Schema relationnel / Relational Schema

PK = Cle primaire | FK = Cle etrangere | Toutes les entites ont id, createdAt, updatedAt

User PK id login, email role actif: Boolean passwordHash	Projet PK id nom, description dateDebut / dateFin FK createdById (!' User)	AuditLog PK id action, entity entityId oldValue / newValue FK userId (!' User)
Mission PK id nom, statut dateDebut / dateFin FK projetId (!' Projet)	Localite PK id nom, commune, region latitude, longitude geom (PostGIS) FK missionId (!' Mission)	Hote PK id nomCommun nomScientifique typeAnimal FK methodId
MethodeCollecte PK id type (PIEGES ASPIR) dispositif dateDebut / dateFin FK localiteId	Moustique PK id genre / espece sexe / stade positionPlaque (A1-H12) FK methodId	Tique / Puce PK id genre / espece / stade positionPlaque FK methodId FK hotelId (optionnel)

Legende : Jaune = Cle primaire (PK) | Cyan = Cle etrangere (FK) | Blanc = Attribut standard

3.4 Conventions de nommage / Naming Conventions		
Element	Convention	Exemple
Modeles Prisma	PascalCase	Moustique, MethodeCollecte
Tables SQL	snake_case (@@map)	moustiques, methodes_collecte
Colonnes	camelCase Prisma / snake_case SQL	createdAt !' created_at
Cles primaires	Int autoincrement	id Int @id @default(autoincrement())
Timestamps	createdAt / updatedAt	@default(now()) / @updatedAt
Relations	Relation nommee + @relation	methode MethodeCollecte @relation(...)

4. Architecture de l'API

4. API Architecture

4.1 Principes generaux / General Principles

- Prefixe global : /api/v1/ — versioning dans l'URL pour compatibilite future
- Toutes les reponses sont en JSON avec enveloppe standard { data, meta, error }
- Authentification par Bearer token JWT dans l'en-tete Authorization
- Global prefix: /api/v1/ — URL versioning for forward compatibility
- All responses are JSON with standard envelope { data, meta, error }
- Authentication via Bearer JWT token in the Authorization header

4.2 Endpoints par ressource / Endpoints by Resource

Methode	Endpoint	Description	Role min.
GET	/api/health	Health check public	public
POST	/api/v1/auth/login	Connexion — retourne JWT	public
POST	/api/v1/auth/register	Creation compte (actif:false)	public
GET	/api/v1/auth/me	Profil utilisateur courant	lecteur
GET	/api/v1/projets	Liste des projets	lecteur
POST	/api/v1/projets	Creer un projet	chercheur
GET	/api/v1/projets/:id	Detail projet + statistiques	lecteur
PUT	/api/v1/projets/:id	Modifier un projet	chercheur
DELETE	/api/v1/projets/:id	Supprimer un projet	admin
GET	/api/v1/missions	Liste missions (filtrables)	lecteur
POST	/api/v1/missions	Creer une mission	chercheur
GET	/api/v1/missions/:id	Detail mission + stats	lecteur
GET	/api/v1/localites	Localites avec coords GPS	lecteur
POST	/api/v1/localites	Creer une localite	terrain
GET	/api/v1/methodes	Methodes de collecte	lecteur
POST	/api/v1/specimens/moustiques	Creer un moustique	terrain
GET	/api/v1/specimens/moustiques	Lister moustiques (filtres)	lecteur
POST	/api/v1/specimens/moustiques/import	Import Excel moustiques	chercheur
GET	/api/v1/specimens/moustiques/export	Export Excel moustiques	chercheur
POST	/api/v1/specimens/tiques	Creer une tique	terrain
POST	/api/v1/specimens/puces	Creer une puce	terrain
GET	/api/v1/search	Recherche globale multi-entite	lecteur
GET	/api/v1/stats	Statistiques tableau de bord	lecteur
GET	/api/v1/notifications	Stream SSE notifications	lecteur
GET	/api/v1/audit	Journal d'audit (pagine)	admin
GET	/api/v1/users	Liste utilisateurs	admin
PUT	/api/v1/users/:id/activate	Activer un compte	admin
PUT	/api/v1/users/:id/role	Changer le role	admin

4.3 Format de reponse standard / Standard Response Format

Succes / Success

```
{ "data": { ... }, "meta": { "total": 42, "page": 1, "limit": 20 } }
```

Erreur / Error

```
{ "error": { "code": "UNAUTHORIZED", "message": "Token invalide ou expire" } }
```

4.4 Chaine de middlewares Express / Express Middleware Chain

Middleware	Role FR	Role EN
<code>cors()</code>	Autorise CLIENT_URL (.env)	Allows CLIENT_URL from .env
<code>helmet()</code>	En-tetes de securite HTTP	HTTP security headers (11 headers)
<code>express.json()</code>	Parse le corps JSON	Parses JSON request body
<code>auth.middleware.js</code>	Verifie le JWT Bearer	Verifies JWT Bearer token
<code>requireRole(...)</code>	Verifie le role exact	Checks exact role match
<code>requireMinRole(...)</code>	Verifie le role minimum	Checks minimum role level
<code>asyncHandler()</code>	Capture les erreurs async	Catches async errors transparently
<code>errorHandler (global)</code>	Formate toutes les erreurs	Formats all error responses as JSON
SSE middleware	Maintient connexions ouvertes	Keeps SSE connections alive with heartbeat

4.5 Import/Export Excel

- POST /import — Lecture positionnelle des colonnes (col 1 = genre, col 2 = espece...) — la ligne d'en-tete est ignoree
- GET /export — Generation en memoire du fichier .xlsx via ExcelJS, retourne avec Content-Disposition: attachment
- POST /import — Positional column parsing (col 1 = genus, col 2 = species...) — header row is skipped
- GET /export — In-memory .xlsx generation via ExcelJS, returned as Content-Disposition: attachment

5. Securite & Authentification

5. Security & Authentication

5.1 Strategie d'authentification / Authentication Strategy

L'application utilise l'authentification sans etat (Stateless) basee sur JSON Web Tokens (JWT). Il n'y a pas de session cote serveur. Le token est genere a la connexion, signe avec JWT_SECRET, et presente a chaque requete protegee.

Parametre	Valeur / Value	Note
Algorithme	HS256	HMAC SHA-256
Expiration	7 jours / 7 days	Configurable dans auth.controller.js
Transport	Authorization: Bearer <token>	Header HTTP standard
Stockage client	localStorage (Zustand persist)	Migrer vers httpOnly cookie en prod
Secret	JWT_SECRET (variable .env)	Jamais committe dans Git / Never in Git
Evolution suggeree	Refresh token + rotation	Access token 15 min + refresh httpOnly

5.2 Flux d'authentification / Authentication Flow

1	Client POST /api/v1/auth/login {login, password}
2	Serveur bcrypt.compare(password, hash) !' jwt.sign(payload, JWT_SECRET) !' { token, user }
3	Client stocke token dans Zustand (localStorage) !' Axios intercepteur ajoute Bearer header
4	Serveur auth.middleware.js extrait header !' jwt.verify() !' req.user = payload
5	Guard requireMinRole(role) compare req.user.role au niveau minimum requis
6	Sur 401 !' Axios intercepteur !' logout() !' redirect /login automatique

5.3 Matrice de controle d'accès RBAC / RBAC Access Control Matrix

Action	lecteur	terrain	chercheur	admin
Consulter donnees	oui	oui	oui	oui
Creer specimen	non	oui	oui	oui
Modifier specimen	non	non	oui	oui
Import Excel	non	non	oui	oui
Export Excel	non	non	oui	oui
Supprimer donnees	non	non	non	oui
Gerer utilisateurs	non	non	non	oui
Voir audit log	non	non	non	oui
Activer comptes	non	non	non	oui

5.4 Mesures de securite implementees / Implemented Security Measures

- Helmet.js — 11 en-tetes de securite HTTP (X-Frame-Options, CSP, HSTS, X-Content-Type-Options...)
- CORS configure sur CLIENT_URL uniquement — bloque toutes les origines non autorisees
- Mots de passe haches avec bcrypt (cost factor recommande >= 12)
- Audit trail automatique — toutes les mutations dans audit_logs avec old/new values (JSON)
- Variables sensibles dans .env — jamais committees dans le depot Git
- Helmet.js — 11 HTTP security headers blocking clickjacking, XSS, MIME sniffing...
- CORS locked to CLIENT_URL — blocks all unauthorized origins
- Passwords hashed with bcrypt (recommended cost factor >= 12)
-

5.5 Améliorations recommandées / Recommended Security Improvements

Amélioration	Description	Priorité
Refresh Token + Rotation	Access token 15 min + refresh token httpOnly cookie	HAUTE
Rate Limiting	express-rate-limit sur /auth/login (anti-brute force)	HAUTE
HTTPS obligatoire	TLS via Nginx + Let's Encrypt ou certificat IPM	HAUTE
httpOnly Cookie	Migrer localStorage vers cookie httpOnly (anti-XSS)	MOYENNE
Validation entrées (Zod)	Schema Zod sur tous les body de requêtes	MOYENNE
Alertes audit temps réel	Notifier admin sur actions sensibles en SSE	OPTIONNELLE
MFA (2FA)	TOTP (Google Authenticator) pour les admins	OPTIONNELLE

6. Scalabilite & Deploiement

6. Scalability & Deployment

6.1 Architecture de deploiement actuelle / Current Deployment Architecture

Deploiement NAS Synology — Production Actuelle / Current Production NAS

NAS Synology !' Docker Compose !' Nginx (80/443) !' Backend Node.js (:3000)
PostgreSQL 15 + PostGIS !' Docker volume persistant / persistent Docker volume
Acces interne IPM : reseau local LAN | Acces externe : Tailscale VPN (en cours / pending)
SSL/TLS : Let's Encrypt via Nginx | Proxy : /api/* !' backend, /* !' React dist (statique)

6.2 Configuration Docker Compose

Service	Image	Port	Role
postgres	postgres:15-alpine	5435:5432	Base de donnees principale
backend	node:22-alpine (custom)	3000:3000	API REST Express.js
frontend	nginx:alpine + build Vite	80:80 / 443:443	Serveur statique + reverse proxy

Le fichier docker-compose.prod.yml orchestre les 3 services avec healthchecks et restart: unless-stopped.

6.3 Procedure de deploiement / Deployment Procedure

1	git pull origin master !' recuperer les dernieres modifications
2	cd frontend && npm run build !' generer le bundle Vite dans dist/
3	docker-compose -f docker-compose.prod.yml build !' reconstruire les images
4	npx prisma migrate deploy !' appliquer les migrations de BDD en production
5	docker-compose -f docker-compose.prod.yml up -d !' redemarrer les conteneurs
6	curl https://sm.ipmnas.synology.me/api/health !' verifier {"status":"ok"}

6.4 Plan de Migration Cloud — Vision Future / Cloud Migration Roadmap

Roadmap de migration vers une infrastructure cloud professionnelle pour la haute disponibilite et l'ouverture a d'autres institutions.

Phase	Horizon	Description	Infrastructure suggeree
Phase 1 Actuelle	Maintenant	NAS Synology + Docker Compose + Tailscale VPN	NAS Synology DS923+
Phase 2 Cloud Hybride	6-12 mois	VPS dedie + CI/CD automatise + monitoring + HTTPS	OVH VPS Comfort (EU)
Phase 3 Cloud Native	12-24 mois	Kubernetes (K3s) + PostgreSQL manage + CDN + WAF	OVH Managed Kubernetes
Phase 4 Multi-institution	24+ mois	Multi-tenant SaaS + API Gateway + IAM federe	AWS / Azure + WAF

Recommandation : OVH Cloud (souverainete des donnees europeenne) pour les donnees de recherche medicale.
Recommendation: OVH Cloud (EU data sovereignty) for sensitive medical research data hosting.

6.5 Application Mobile Terrain — Architecture Future / Field Mobile App

Pour la saisie terrain en conditions difficiles (hors connexion, GPS, photos specimens) :

Composant	Technologie	Justification
App framework	React Native + Expo	Partage de code avec le frontend React existant
Navigation	Expo Router	File-based routing, coherent avec React Router v7
Stockage offline	WatermelonDB + SQLite	Base locale haute performance pour sync differee
Synchronisation	Queue de mutations + retry	Envoi automatique quand connexion disponible
GPS terrain	expo-location	Capture coordonnees GPS automatiquement
Photos specimens	expo-camera	Documentation photographique des specimens
Authentification	JWT existant reutilise	Meme backend — zero duplication de code
Build & OTA	EAS Build + EAS Update	Mises a jour sans passer par les stores

6.6 Fonctionnalites Suggerees / Suggested Future Features

Fonctionnalite	Description	Valeur
Analyse statistique	Graphiques abondance, tendances, heatmaps GPS	TRES HAUTE
Module photos	Photos specimens avec metadonnees EXIF + MinIO	TRES HAUTE
Export PDF rapports	Rapports de mission en PDF (mission + specimens)	TRES HAUTE
Dashboard analytics	BI interne — tendances, alertes epidemio	HAUTE
Cartographie avancee	Couches SIG + zones Fokontany interactives	HAUTE
Validation taxonomique	Integration GBIF / iNaturalist (externe)	HAUTE
Multi-institution	Isolation par tenant (schema PostgreSQL)	MOYEN TERME
API publique	API ouverte rate-limitee pour partenaires	LONG TERME

Synthese & Points Cles

Summary & Key Takeaways

Forces de l'architecture actuelle / Current Architecture Strengths

- Stack PERN eprouvee, bien documentee, large communaute de developpeurs
- Prisma ORM : migrations versionnees, type safety end-to-end TypeScript
- Hierarchie de roles graduee (4 niveaux) adaptee au contexte institutionnel
- Audit trail automatique sur toutes les mutations — traçabilité complete
- Architecture entierement conteneurisee : reproductible et portable
- PostGIS integre : pret pour analyses geospatiales avancees (fokontany, zones)
- TanStack Query : cache intelligent, revalidation automatique, UX optimale

Points d'attention prioritaires / Priority Action Items

- SECURITE : Migrer le JWT de localStorage vers httpOnly cookie (protection XSS)
- SECURITE : Implementer rate-limiting sur /auth/login (protection brute force)
- QUALITE : Ajouter suite de tests Jest + Supertest avant toute montee en charge
- DEVOPS : Mettre en place CI/CD GitHub Actions pour deployments automatises
- MOBILE : Initier le PoC React Native + Expo pour la saisie terrain offline
- ANALYTICS : Implementer le module statistiques pour valoriser les donnees collectees
- INFRA : Activer HTTPS Let's Encrypt via Nginx en production NAS

Roadmap recommandee 12 mois / Recommended 12-Month Roadmap

Trimestre	Priorites / Priorities
T3 2026 Jul-Sep	Rate limiting + httpOnly cookie Tests Jest/Supertest CI/CD GitHub Actions HTTPS
T4 2026 Oct-Dec	Module photos specimens (MinIO) Export PDF rapports Dashboard analytics complet
T1 2027 Jan-Mar	PoC app mobile React Native + Expo Sync offline Migration vers VPS OVH Cloud
T2 2027 Avr-Jun	App mobile production Validation taxonomique GBIF Multi-institution (beta)

Contact & Informations projet / Project Information

Institut Pasteur Madagascar — SpécimenManager v1.0

Responsable technique : Henintsoa

Institution : Institut Pasteur Madagascar (IPM)

Document genere le : 28 Juin 2026 — Claude Code (Anthropic Sonnet 4.6)

Depot Git : github.com/IPM / specimenmanager (branche master)

Ce document est confidentiel et destine a un usage interne a l'Institut Pasteur Madagascar.
Toute reproduction ou diffusion externe requiert l'autorisation explicite de la direction.
This document is confidential and intended for internal use at Institut Pasteur Madagascar only.
Any reproduction or external distribution requires explicit management authorization.